Column counts in MULTIZ: `yama` no longer costs $K \times L$ per cell

Richard Burhans
Penn State University ORCID 0009-0006-5949-0958

Technical note, 5 November 2026

Abstract

MULTIZ builds genome-wide multiple alignments by repeatedly aligning one alignment to another. Its dynamic programming routine, `yama`, scored every cell by summing over all $K \times L$ pairs of rows, where K and L are the depths of the two blocks. On one deep merge from a 217-genome alignment, `yama` took 98 percent of the run time. Release 11.4 scores each cell from summaries made once per column instead: a score per symbol for one block and symbol counts for the other, plus counts of four gap classes. The work per cell no longer depends on the depths. Every score is an integer, so each total is the same as the pair-by-pair sum, and the output does not change. On synthetic blocks of $K = L = 128$ rows, `yama` itself ran about a thousand times faster and the whole merge 63 times faster, with identical output. On these blocks the rest of the time is now spent scoring the merged blocks, which still visits every row pair. The idea is not new. Scoring a column from symbol counts is a profile, Gotoh counted gap states per column to make aligning groups of sequences much less dependent on their sizes, and Kececioglu and Zhang aligned two alignments under this gap model with work per cell that does not grow with their depth. Scoring columns from symbol counts came from profile aligners, and keeping the gap sums equal to 11.3's came from reading how `yama` charges gaps. A profile of one deep merge showed where it would pay. We found the papers of Gribskov and colleagues, Gotoh, and Kececioglu and Zhang only afterwards. This note records how the change was found, what it computes, where it comes from, and what it measured.

1 The problem

MULTIZ and the threaded blockset aligner build a multiple alignment progressively along a guide tree, and each step aligns two alignments that share a reference sequence [2]. The routine that does it, `yama` in `mz_yama.c`, runs a banded dynamic programming over the columns of the two blocks. Its objective is the sum of pairs: a column of block A (with K rows) aligned to a column of block B (with L rows) scores the sum of the substitution scores of all $K \times L$ row pairs. Gaps are charged by Altschul's quasi-natural gap costs [1], on the graph model of Myers and Miller [12] as Chao, Hardison and Miller modified it for those costs [3]. Release 11.3 computed each cell's substitution sum and its three gap sums with loops over all $K \times L$ row pairs.

That cost grows with the product of the depths. Deep blocks are what a large panel produces. In a 217-genome alignment of *Cannabis sativa* assemblies, cs10 and 216 others, one merge aligned 2,303 blocks of about 35 rows to 4,359 blocks of about 43 [13]. The pull request counts the 216 queries. In 11.3, every cell visited each pair of rows it aligned, four times. Its caller, `pre_yama`, sets the shared cs10 row aside, so the first call aligns one row fewer from each block, and a second call aligns cs10 to the result. The pull request records that `yama` took 98 percent of that merge's

run time [13]. That profile was kept: a `gprof` build of 11.3 spent 98.1 percent of 950 s in `yama` (`measurements/gprof_deepmerge_v11.3.txt`). The host and its other load were not recorded. Scoring columns from symbol counts came from profile aligners, and keeping the gap sums equal to 11.3's came from reading how `yama` charges gaps. The profile showed that `yama` was where it would pay.

2 The change

A row enters a cell's sums only through two things: its symbol in the current column, and whether it and the row's entry in the previous column are dashes. So each sum can be rebuilt from counts of those classes (Fig. 1).

Substitution. For each column of A, release 11.4 computes once a score for every symbol s,

$$w_A(s) = \sum_{i=1}^{K} \sigma(a_i, s),$$

where σ is the substitution matrix. For each column of B it counts the rows holding each symbol, $n_B(s)$. The cell's substitution sum is then $\sum_s n_B(s)\, w_A(s)$, a dot product over the symbols that occur anywhere in block B (five in the synthetic blocks below), the dash included, and symbols absent from this column count zero. The dash matters: a base against a dash is where `yama` charges gap extension, so a version that counted only A, C, G, T and N would not be exact. Preparing w_A costs K steps per column of A for each symbol, and n_B costs L per column of B. Neither is repeated per cell, and the number of symbols is bounded by the alphabet, not by the depth.

Gaps. Under quasi-natural gap costs, whether a pair of rows opens a gap at a new column is decided by the preceding column alone. Altschul defined them that way, adding one gap in the case where natural costs would need more history, so that "the preceding edge completely determines the number of gaps a new edge introduces" [1]. A row's contribution to the gap sums therefore depends only on two flags: is it a dash here, and was it a dash in the previous column. Each row falls into one of four classes, and the gap sums are products of class counts from the two blocks.

Exactness. All scores are integers. Each total is the pair-by-pair sum regrouped, so it is the same number, and the traceback makes the same choices. The output of 11.4 is identical to that of 11.3 apart from the `#` header lines (Section 3).

3 What it measured

Synthetic blocks. To measure the scaling on public, reproducible input, we built 11.3 and 11.4 from their release tags with identical flags (`-O2`) and merged synthetic block pairs with $K = L$ from 2 to 128. Each pair has 10 blocks of 2,000 reference columns, a shared reference row, species rows with 2 percent substitutions and 1 percent deletions, and insertion columns placed differently in the two files, so `yama` has columns to align. Times are medians of three runs (Table 1, Fig. 2). The 11.3 time rose about four times for each doubling of K, as $K \times L$ predicts, from 0.34 s at $K = 8$ to 76.7 s at $K = 128$. The 11.4 time stayed between 0.06 and 0.13 s up to $K = 32$ and reached 1.22 s at 128. The speed-up was 5.4-fold at $K = 8$, 37-fold at 32 and 63-fold at 128. Outputs were identical at all seven depths.

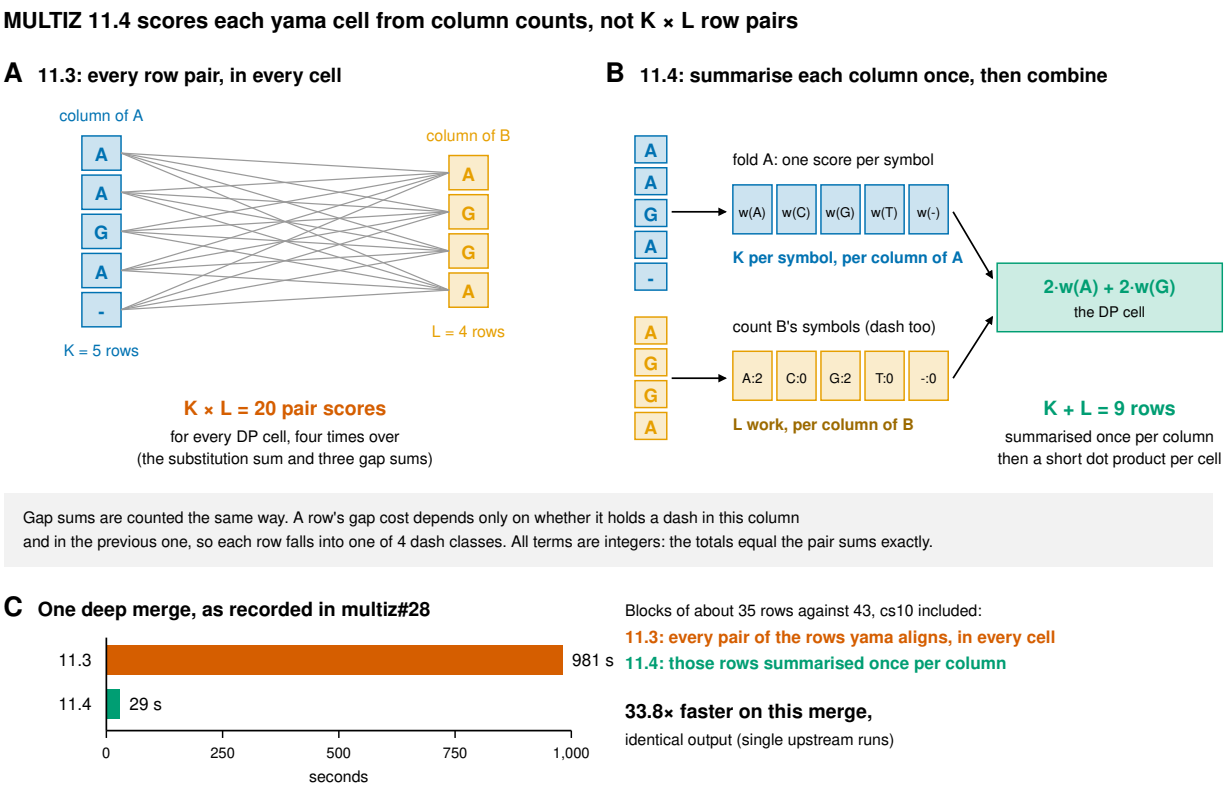


Figure 1: Column counts turn $K \times L$ work per cell into $K + L$ work per column. In panel A, release 11.3 scored a toy column pair of $K = 5$ and $L = 4$ rows by visiting all 20 row pairs, once for the substitution sum and once for each of three gap sums. In panel B, release 11.4 folds the column of A into one score per symbol, at a cost of K per symbol, and counts the symbols of B's column, the dash included, each once per column. The cell is their dot product, here $2w(\mathrm{A}) + 2w(\mathrm{G})$. Gap sums are counted by four dash classes. Blue marks A, orange B, green the 11.4 cell and vermilion the 11.3 row-pair work. Panels A and B are a toy example, not data. Panel C is the deep merge recorded in `multiz#28`, one run per release, with blocks of about 35 and 43 rows.

Product, not square. With $K = L$ the sweep cannot tell $K \times L$ from K^2. A grid of unequal depths, K and L each 4, 16 or 64, separates them (Fig. 3). Over this grid, pairs with the same $K \times L$ took 11.3 times within about 20 percent of each other. At $K \times L = 64$ the 4×16 and 16×4 pairs took 0.33 and 0.39 s. At 256, the 4×64, 16×16 and 64×4 pairs took 1.27, 1.25 and 1.46 s. At 1,024 the 16×64 and 64×16 pairs took 4.83 and 4.80 s, against 4.84 s for 32×32. The shape is not irrelevant. On identical files, putting the deeper alignment first made 11.3 18 to 20 percent slower at 4×16 and 12 to 15 percent slower at 4×64, and no difference was detected at 16×64 (medians within 1 percent). Each run is a median of five per order, with four runs at 4×16 and 4×64, three of them in one session, and three at 16×64 (`measurements/swap_order*.csv`). In these three shapes the order mattered only when one side had 4 rows. Shapes thinner than the grid's were not measured here. All nine outputs were identical.

Where the time goes now. The 11.4 time does not stay proportional to $K + L$ at depth. From $K = L = 64$ to 128 it rose 3.8-fold. A `gprof` profile of the $K = L = 128$ merge shows why (`measurements/gprof_k128.txt`). In 11.3, `yama` took 76.7 s of 77.8 and `mafScoreRange` 1.1 s. In 11.4, `yama` took 0.06 s, about a thousand times less, while `mafScoreRange` still took 1.1 s, now 94 percent

Table 1: Synthetic block pairs, median of three runs per release. Identical: the two outputs agree after removing # header lines.

$K = L$	$K \times L$	v11.3 (s)	v11.4 (s)	speed-up	identical
2	4	0.057	0.061	0.93	yes
4	16	0.113	0.059	1.92	yes
8	64	0.340	0.063	5.37	yes
16	256	1.282	0.076	16.84	yes
32	1,024	4.842	0.129	37.45	yes
64	4,096	19.412	0.322	60.35	yes
128	16,384	76.733	1.220	62.92	yes

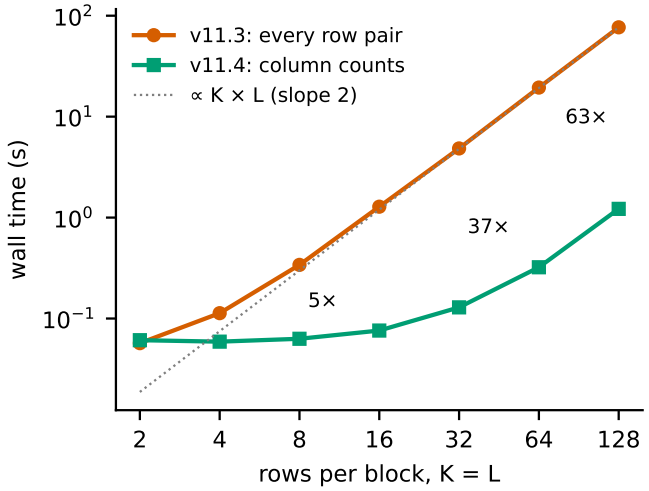


Figure 2: The 11.3 time follows $K \times L$, and the 11.4 time grows far more slowly. Wall time of one `multiz` merge of a synthetic block pair against block depth, log-log, median of three runs. Vermilion circles are v11.3, green squares v11.4, and the dotted line has slope 2 through the last v11.3 point. Labels give the speed-up. Below $K = 4$ both releases spend their time outside `yama`.

of the run. `mafScoreRange` scores each merged block by summing over all pairs of its $K + L - 1$ rows, so it grows with the square of the total depth. The speed-up therefore levels off in this sweep, at 60-fold at $K = 64$ and 63-fold at 128, and it is lower for a deep block merged with a shallow one. Counting would also make `mafScoreRange` linear in depth and exact, in the sense that it would equal the function's own pair-by-pair sum. Symbols and the four dash classes are counted per column as in `yama`, but its pairs lie within one block. So each class is also paired with itself, each row's pair with itself is left out, and each unordered pair is counted once. Its gap model also looks back one column, so per-column counting is again the pessimistic count of Kececioglu and Zhang [7]. Ranwez computes the substitution part from per-site counts, and even the exact affine sum-of-pairs score, in time linear in the number of rows times the number of columns [10]. On real merges, too, `mafScoreRange` is now the larger cost in deep merges. A `gprof` build of 11.4 spent 63 percent of the deep merge below in `mafScoreRange` and 35 percent in `yama` (`measurements/gprof_deepmerge_v11.4.txt`). That makes it the next target for deep merges, though removing it entirely would cut that merge's sampled `multiz` time at most about 2.7-fold.

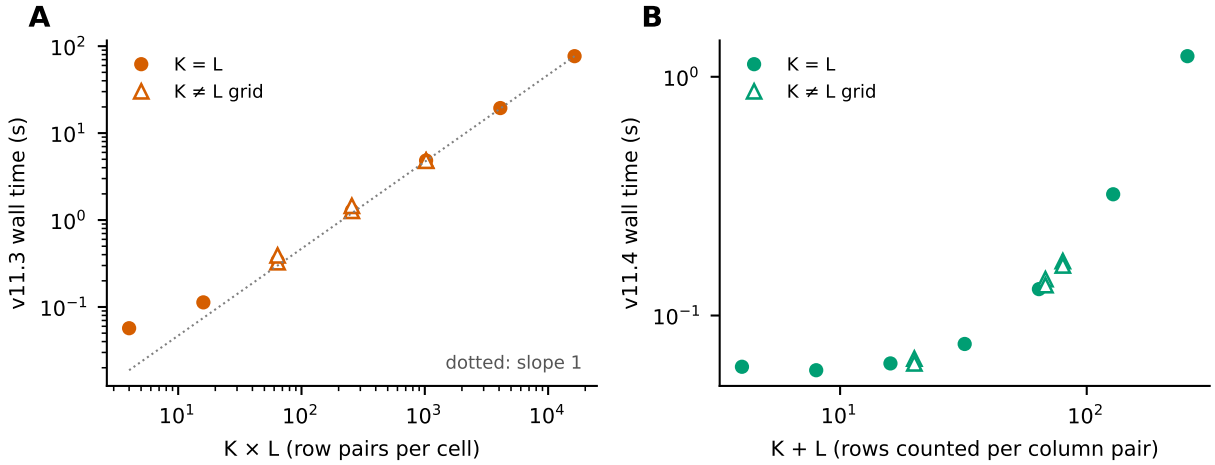


Figure 3: The 11.3 time follows $K \times L$. Panel A plots the v11.3 wall time against $K \times L$ and panel B the v11.4 time against $K + L$, both log-log. Filled circles are the $K = L$ sweep, open triangles the grid with $K \neq L$ (each of 4, 16 and 64). The dotted line in A has slope 1. Medians of three runs, all outputs identical. In B the time is flat at about 0.06 s for small depths, where little time is spent aligning, and grows faster than $K + L$ at large depths, where `mafScoreRange` dominates.

Real merges. The pull request that introduced the change records three more measurements [13]. The deep merge above took 981 s with the 11.3 release build and 29 s with 11.4, one run each on a host whose other load was not recorded, and its output and both files of unused blocks were identical apart from the header line. A whole 217-genome roast of one 4.4 Mb *Cannabis* piece took a median 14,085 s with 11.3 and 2,873 s with 11.4 (4.9-fold, 4.5- to 5.2-fold across single-run pairings), three interleaved runs per release on one host, all six outputs identical apart from # lines. UCSC's hg38 `multiz5way` recipe still reproduced the published alignment, 7 of 7 sequences.

Where a whole run's time goes. One deep merge gained 33.8-fold and a whole piece 4.9-fold. A `gprof` build of 11.4 ran the whole piece, all 215 merges, with output identical to the production alignment apart from # lines. It spent 53 percent of the sampled `multiz` time in `mafScoreRange` and 45 percent in `yama` (`measurements/gprof_wholepiece_v11.4.txt`). A second run agreed within 0.1 percentage points, and a third, from the recipe in Methods, gave 54 and 43 percent. Per merge, from the second run (`measurements/wholepiece_permerge_v11.4.tsv`, one row per `multiz` process in the order they ran), `mafScoreRange` led in 29 of the 215 merges and `yama` in the other 186. All 29 are among the last 36 of the 215 merges to run. These shares cover `multiz` alone. `roast` also runs `maf_project`, which calls `mafScoreRange` too and was not profiled, and how the rest of the wall time divides, including library and system time that `gprof` does not sample, was not measured.

4 Prior work

The idea is not new.

Scoring a column from symbol counts is a profile. Gribskov, McLachlan and Eisenberg defined the profile score of symbol a at position p as $M(p, a) = \sum_b W(p, b)\, Y(a, b)$, with Y the substitution matrix and the simple weight $W(p, b)$ the count of residue b in that column divided by the number of sequences [6]. The fold w_A above is the same sum without the division, with the dash scored as

one more symbol. Between two groups of sequences, profile operations of this kind are Gotoh's algorithms A and B [4].

Counting gap states per column to escape the $K \times L$ loop is Gotoh's. Gotoh gave rigorous sum-of-pairs algorithms for aligning two groups of sequences [4], and then made their cost much less dependent on the product of the group sizes. In his words, profile vectors could already reduce the work, but "there has been no way to evaluate the accurate gap-opening costs with computation steps less than $O(M \times N)$" [5]. He did it with gap-state variables computed once per column and combined during the recursion, and reported speed-ups over his earlier algorithm of about 10-fold when $M \times N \approx 200$ and over 100-fold when $M \times N > 2{,}500$. His gap states record runs of nulls of any length, because his objective charges every gap opening. `yama`'s quasi-natural costs look back one column, so the states reduce to four classes. Kececioglu and Zhang already rewrote their recurrences in terms of these four per-column counts, f_{00}, f_{01}, f_{10} and f_{11}, where f_{xy} counts rows whose entries in the previous and current columns are a character (1) or a dash (0). For the boundary conditions they add a count of rows whose first character falls in the column. They noted that under optimistic or pessimistic counts, "by storing just four additional integers at each position in a profile", profiles "become equivalent in descriptive power to the original multiple alignment" [7].

For quasi-natural gap counts, which Kececioglu and Zhang call pessimistic, the combination is not new either. They gave algorithms that align two alignments under them in essentially $O(mn)$ time [7]. Their Theorem 4 gives $O(km + \ell n + \min\{s^2, k\ell\}\, mn)$ for alignments of k and ℓ rows and lengths m and n over an alphabet of size s [7]. Wheeler and Kececioglu state the bound as $O(kn + n^2)$ [11], which is Theorem 4's for a fixed alphabet with $k = \ell$ and $m = n$. Release 11.4 computes that objective, up to `yama`'s free end gaps. We did not take it from their paper, which we found only afterwards and have since read, apart from the appendix that holds the remaining recurrences. Their paper is a conference paper that omits its proofs, with recurrences and time bounds but no timing or accuracy measurements. It reports one application of its sequence-against-alignment algorithm, whose study is in a technical report.

Exact gap counting is what makes the general problem hard. With linear gap costs counted exactly, aligning two alignments under the sum-of-pairs objective is NP-complete [8, 9]. Release 11.4 does not touch that problem. It computes `yama`'s existing objective, quasi-natural gaps included, exactly and faster. That objective is itself an approximation. Kececioglu and Starrett found the pessimistic recurrence's own score up to 1.02 percent and its gap count up to 31.63 percent from the exact optimum (0.52 and 20.04 percent for the alignment it returns), on four 12-sequence protein benchmarks, each split every way into two alignments (their Tables 3 and 4). All four maxima are on rh12, whose 20.04 percent is the error they report on "the toughest benchmark". There the means were 0.78 and 16.51 percent for the recurrence and 0.04 and 0.63 percent for the alignment it returns. We know of no measurement of this error for DNA or under `yama`'s scores, which charge 400 to open a gap, about four times a match, and we did not search for one. Within a progressive aligner that merges along a minimum spanning tree, Wheeler and Kececioglu found pessimistic merging nearly as accurate as exact merging on three protein benchmark suites, BAliBASE (up to 40 sequences), SABmark (up to 25) and PALI (at least 7). Their SPS accuracy, averaged over the three suites, was 71.6 for exact merging and 70.9 for pessimistic merging [11]. Release 11.4 inherits that approximation unchanged.

A search on 5 November 2026 (`measurements/prior_art_search.md`) found no publication that applies this reduction to MULTIZ or the threaded blockset aligner. It was narrow: one open-ended query and no forward-citation search, it missed Kececioglu and Zhang's result until a review found it, and it did not read Chao, Hardison and Miller [3] or the MultiPipMaker papers, which also use quasi-natural gaps.

5 Limits

- The sweep's blocks are synthetic: uniform random sequence, fixed mutation rates, one seed per shape, and only A, C, G, T and the dash. Real soft-masked input adds lower case and N, which give 11.4 more symbols to count. The blocks show how the cost scales with depth, not how a real alignment behaves.
- The sweep ran on a 32-vCPU host while unrelated single-threaded alignment jobs were running (load average about 4.8). Each `multiz` run is single-threaded, and the two releases ran back to back for each size.
- The profiles are `gprof` samples at 0.01 s, so the 0.06 s for `yama` in 11.4 is coarse. It supports "about a thousandfold", not a precise ratio.
- The deep-merge and whole-piece timings are taken from the pull request and were not rerun for this note. The profiles come from `-pg` builds, so their shares are of sampled CPU time in `multiz`, not of wall time. The whole-piece and 11.4 deep-merge profiles ran on a host with other load. The deep merge is one run per release, on a host whose other load was not recorded.
- Output identity was checked on the seven synthetic pairs, the deep merge, the whole piece and the hg38 5-way recipe. It is an empirical check of the exactness argument, not a proof.

Methods and availability

The script `probe/build_multiz.sh` clones `multiz` at tags `v11.3` (commit `645ab55`) and `v11.4` (`6bfe6a4`) and builds both with `-O2 -Wall -Wextra` (gcc 15.2.0). The script `probe/make_blocks.py` writes the synthetic pairs (seed 1), and `probe/swap_order.py` times one pair in both argument orders. The script `probe/run_sweep.py` times them and compares outputs, writing `measurements/sweep.csv` ($K = L$) and `measurements/grid.csv` (`--sizes 4x4,4x16,...,64x64`). The script `probe/profile_k128.sh` builds both releases with `-pg` and writes `measurements/gprof_k128.txt`. The file `sweep.csv` was written by an earlier revision of `run_sweep.py` that named its inputs `k{K}` instead of `k{K}l{L}`, and the inputs are byte-identical under both names. The file `gprof_k128.txt` was written by the matching earlier revision of `profile_k128.sh`, which read the old names. The deep-merge profiles (`gprof_deepmerge_v11.3.txt` and `gprof_deepmerge_v11.4.txt`) and the whole-piece profile (`gprof_wholepiece_v11.4.txt`) come with checksums of their inputs (`measurements/*_inputs.sha256`). The script `probe/profile_wholepiece.sh` is the whole-piece recipe, written after the runs. Run once since, it gave the same call counts, shares of 54 and 43 percent, and output identical to production (`gprof_wholepiece_v11.4_rep3.txt`). The host is recorded in `measurements/sweep_env.txt`. Figures are made by `paper/make_figure1.py`, `paper/make_figure2.py` and `paper/make_figure3.py`. MULTIZ is at https://github.com/multiz/multiz. Text, figures and code are CC0 1.0.

Acknowledgements

We thank Minmei Hou for discussion of this change.

References

[1] S. F. Altschul. Gap costs for multiple sequence alignment. *Journal of Theoretical Biology* 138:297–309, 1989. doi:10.1016/S0022-5193(89)80196-1.

[2] M. Blanchette, W. J. Kent, C. Riemer, et al. Aligning multiple genomic sequences with the threaded blockset aligner. *Genome Research* 14:708–715, 2004. doi:10.1101/gr.1933104.

[3] K.-M. Chao, R. C. Hardison and W. Miller. Recent developments in linear-space alignment methods: a survey. *Journal of Computational Biology* 1:271–291, 1994. doi:10.1089/cmb.1994.1.271.

[4] O. Gotoh. Optimal alignment between groups of sequences and its application to multiple sequence alignment. *Computer Applications in the Biosciences* 9:361–370, 1993. doi:10.1093/bioinformatics/9.3.361.

[5] O. Gotoh. Further improvement in methods of group-to-group sequence alignment with generalized profile operations. *Computer Applications in the Biosciences* 10:379–387, 1994. doi:10.1093/bioinformatics/10.4.379.

[6] M. Gribskov, A. D. McLachlan and D. Eisenberg. Profile analysis: detection of distantly related proteins. *Proceedings of the National Academy of Sciences USA* 84:4355–4358, 1987. doi:10.1073/pnas.84.13.4355.

[7] J. Kececioglu and W. Zhang. Aligning alignments. In *Combinatorial Pattern Matching* (*CPM 1998*), Lecture Notes in Computer Science 1448, 189–208, 1998. doi:10.1007/BFb0030790.

[8] J. Kececioglu and D. Starrett. Aligning alignments exactly. In *Proceedings of the Eighth Annual International Conference on Research in Computational Molecular Biology* (*RECOMB 2004*), 85–96, 2004. doi:10.1145/974614.974626.

[9] B. Ma, Z. Wang and K. Zhang. Alignment between two multiple alignments. In *Combinatorial Pattern Matching* (*CPM 2003*), Lecture Notes in Computer Science, 254–265, 2003. doi:10.1007/3-540-44888-8_19.

[10] V. Ranwez. Two simple and efficient algorithms to compute the SP-score objective function of a multiple sequence alignment. *PLOS ONE* 11(8):e0160043, 2016. doi:10.1371/journal.pone.0160043.

[11] T. Wheeler and J. Kececioglu. Multiple alignment by aligning alignments. *Bioinformatics* 23(13):i559–i568, 2007. doi:10.1093/bioinformatics/btm226.

[12] E. Myers and W. Miller. Approximate matching of regular expressions. *Bulletin of Mathematical Biology* 51:5–37, 1989. doi:10.1007/BF02458834.

[13] multiz/multiz pull request 28, "yama: build each cell's sums from column counts, not row pairs" (2026). https://github.com/multiz/multiz/pull/28.